

2.7 🐢 项目一：Turtlesim 通信实战

所属阶段： 第二阶段 · ROS 2 核心通信机制

讲次数量： 11 节

预计时长： ~70 分钟 **前置依赖：** 2.1-2.6 全部内容

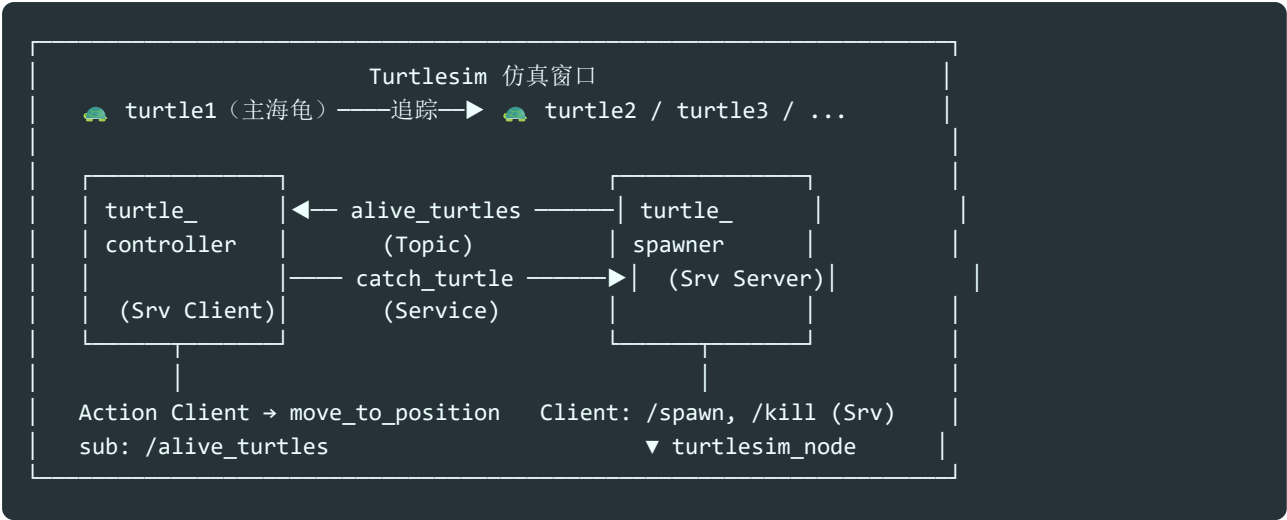
学习目标

完成本项目后，学员将能够：

- 综合运用三种通信机制 (Topic/Service/Action) + Parameter 配置 + Launch 编排
- 从零构建一个多节点协作的 Turtlesim 应用——"Catch Them All" 海龟抓捕游戏
- 按照 6 个 Checkpoint 递进式开发，每步产出可独立运行的完整代码
- 创建自定义接口包 `my_robot_interfaces`，定义 Msg、Srv、Action
- 编写 Launch 文件与 YAML 参数文件，实现一键启动完整系统
- 理解 Python 在 ROS 2 多节点项目中的工程组织方式，并能把 Topic、Service、Action、Parameter 与 Launch 串成完整应用

项目概述

最终效果: "Catch Them All"



通信方向说明:

- `turtle_controller` → `move_to_position_server`: 发送 Action Goal (controller 是 Client, action_server 是 Server)
- `move_to_position_server` → `turtlesim_node`: 发布 `/turtle1/cmd_vel` Topic + 订阅 `/turtle1/pose` Topic
- `turtle_controller` → `turtle_spawner`: 调用 `catch_turtle` 服务 (controller 是 Client, spawner 是 Server)
- `turtle_spawner` → `turtlesim_node`: 调用 `/spawn` 和 `/kill` 服务 (spawner 是 Client)
- `turtle_spawner` → `turtle_controller`: 发布 `alive_turtles` Topic

游戏规则:

- `turtle_spawner` 节点按设定频率调用 `/spawn` 服务, 在窗口中随机位置生成新海龟
- `turtle_controller` 节点控制 `turtle1` 自动追踪并"抓住"目标海龟
- 主海龟到达目标后, 调用 `catch_turtle` 服务消除该海龟
- 消除后 `spawner` 继续生成新海龟, 游戏循环进行
- 所有行为可通过 Parameter 配置: 追踪策略、生成频率、最大存活数量、移动速度等

涵盖的通信机制

机制	本项目中的应用
Topic	<code>/turtle1/cmd_vel</code> 控制运动; <code>/turtle1/pose</code> 监听位姿; <code>alive_turtles</code> 广播存活海龟列表
Service	<code>/spawn</code> 生成海龟; <code>/kill</code> 消除海龟; <code>catch_turtle</code> 自定义抓捕服务
Action	<code>move_to_position</code> 长时间移动任务, 带进度反馈与取消支持
Parameter	<code>catch_closest_turtle_first</code> 、 <code>spawn_frequency</code> 、 <code>max_alive_turtles</code> 、 <code>max_speed</code> 等
Launch	一键启动 <code>turtlesim + controller + spawner + action server</code>

开发环境

- 操作系统: Ubuntu 24.04
- ROS 2 发行版: Jazzy Jalisco (当前 LTS)
- 工作空间: `~/ros2_ws`

安装依赖:

```
sudo apt update  
sudo apt install ros-jazzy-turtlesim ros-jazzy-ros2launch
```

讲次 1：项目准备与 Turtlesim 接口复盘

知识点

本项目不再重复 2.2、2.3 中的 Turtlesim 入门实验，而是把它作为项目运行环境来使用。开始编码前，先确认你能用命令行看到以下接口：

接口	类型	本项目用途	前置讲次
<code>/turtle1/cmd_vel</code>	<code>geometry_msgs/msg/Twist</code>	控制主海龟运动	2.2 Topic
<code>/turtle1/pose</code>	<code>turtlesim/msg/Pose</code>	获取主海龟当前位置	2.2 Topic
<code>/spawn</code>	<code>turtlesim/srv/Spawn</code>	随机生成目标海龟	2.3 Service
<code>/kill</code>	<code>turtlesim/srv/Kill</code>	抓到后删除目标海龟	2.3 Service
<code>/turtle1/rotate_absolute</code>	<code>turtlesim/action/RotateAbsolute</code>	官方 Action 示例，本项目会自定义移动 Action	2.4 Action

项目中最重要 Turtlesim 约束：

- 仿真窗口坐标范围约为 `0.0 ~ 11.0`，生成目标时应避开边界。
- `Twist.linear.x` 控制前进，`Twist.angular.z` 控制平面旋转。
- 速度指令需要持续发布，低频或只发布一次会让运动看起来不明显。
- `/spawn` 和 `/kill` 是服务调用，适合一次性管理海龟生命周期。

快速自检

```
ros2 run turtlesim turtlesim_node
ros2 topic list -t
ros2 service list -t
ros2 interface show turtlesim/msg/Pose
ros2 interface show turtlesim/srv/Spawn
```

若这些命令不熟练，先回到 2.2 的 Turtlesim Topic 实验和 2.3 的 Turtlesim Service 实验复习；本项目从下一讲开始直接进入工程实现。

讲次 2：【LAB】项目接口冒烟测试

实验目标

用最少命令确认仿真器、话题和服务都可用，为后续 Checkpoint 排除环境问题。

Step 1: 启动 Turtlesim

```
source ~/ros2_ws/install/setup.bash
ros2 run turtlesim turtlesim_node
```

Step 2: 确认项目需要的接口存在

```
ros2 topic list -t | grep turtle1
ros2 service list -t | grep -E "spawn|kill"
```

预期至少能看到：

```
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/pose [turtlesim/msg/Pose]
/spawn [turtlesim/srv/Spawn]
/kill [turtlesim/srv/Kill]
```

Step 3: 手动生成并删除目标海龟

```
ros2 service call /spawn turtlesim/srv/Spawn \
  "{x: 5.0, y: 5.0, theta: 1.57, name: 'target_test'}"

ros2 service call /kill turtlesim/srv/Kill "{name: 'target_test'"
```

Step 4: 验证主海龟能被速度话题控制

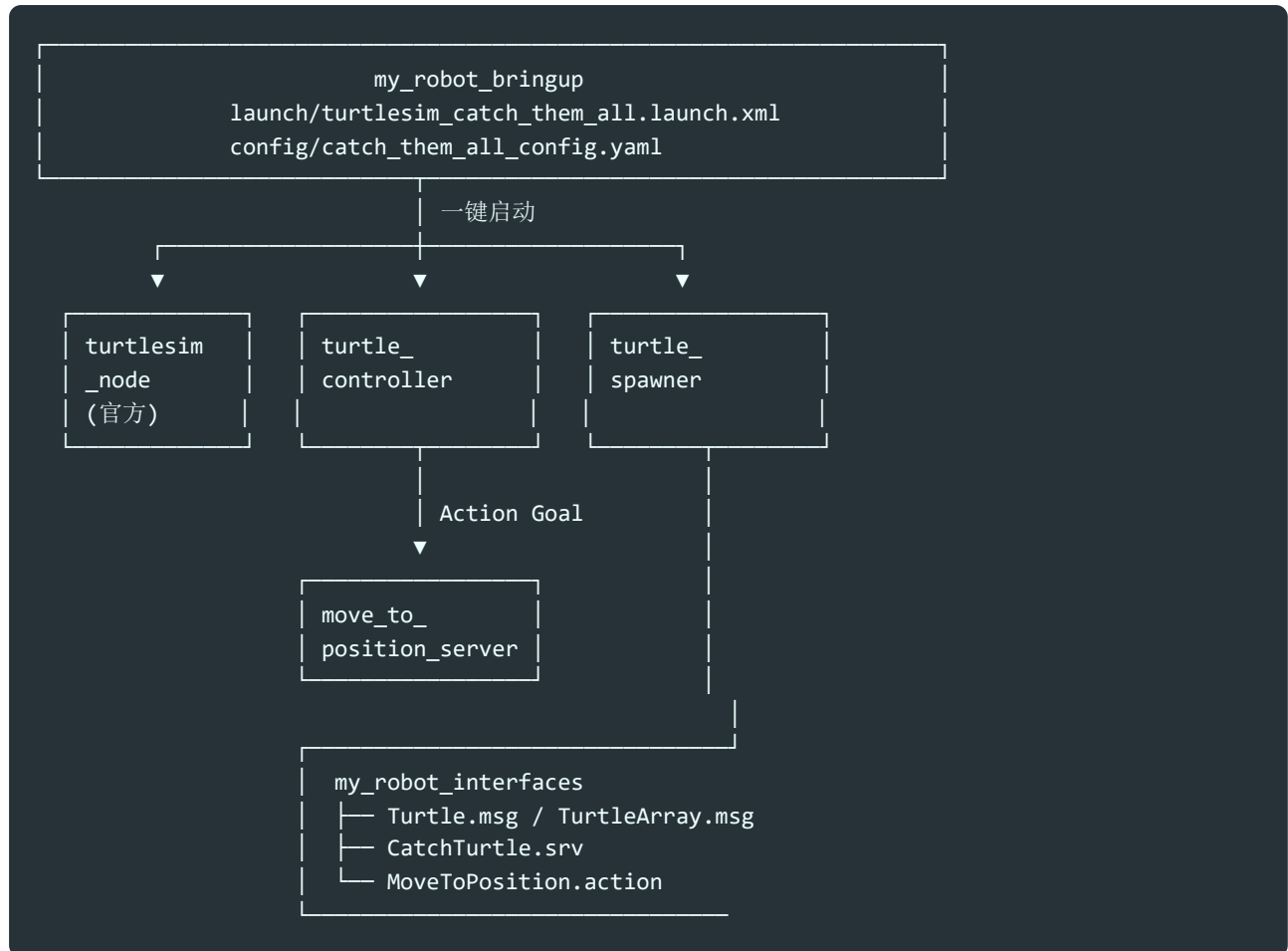
```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 1.0}, angular: {z: 0.5}}" --rate 10
```

看到海龟运动后按 `Ctrl+C` 停止。至此环境可用，后面的 Checkpoint 只关注项目代码。

讲次 3: Turtlesim Project 最终效果预览

系统架构

完成全部 6 个 Checkpoint 后，系统将包含 **4 个节点** 和 **3 个功能包**：



运行最终效果

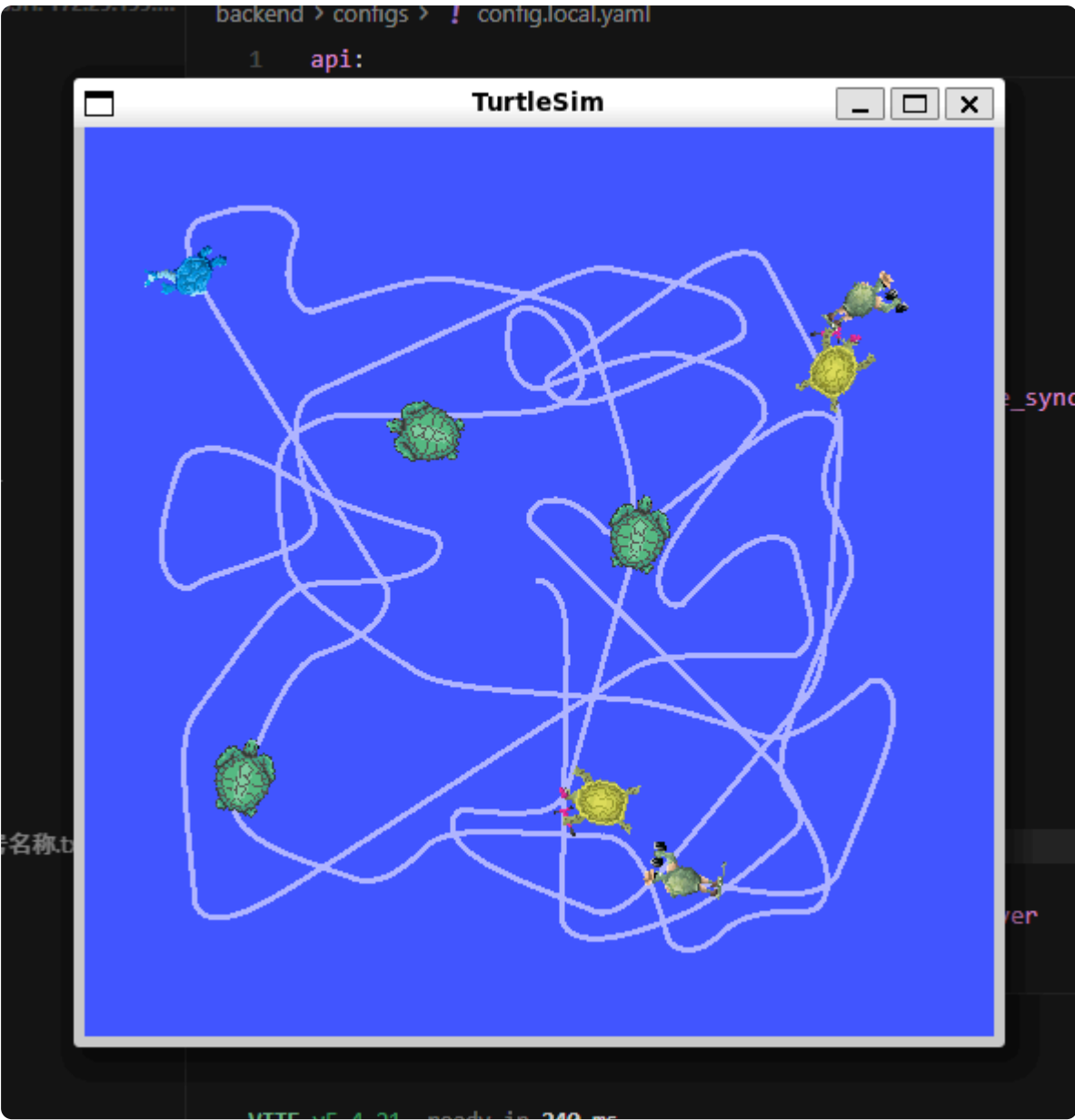
```
# 构建（仅需一次）
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash

# 一键启动
ros2 launch my_robot_bringup turtlesim_catch_them_all.launch.xml
```

预期现象：

- 窗口中按 `spawn_frequency` 参数 (Hz) 生成新海龟，YAML 配置为 1.5 Hz 时约每 0.67 秒尝试生成一只（随机位置）
- 同时存活海龟数量受 `max_alive_turtles` 限制，避免目标过多导致画面和日志失控
- `turtle1` 自动转向并追踪最近的海龟

- 到达目标后目标海龟消失，日志输出 `New alive turtle: ...`
- 修改 YAML 参数可调整生成频率和追踪策略



Checkpoint 路线图

Checkpoint	新增能力	可运行验证
CP1	Topic Publisher — 开环控制运动	海龟画圆
CP2	Topic Subscriber — 闭环追踪固定点	海龟移动到 (2, 8)
CP3	自定义接口 + Service Client 生成多海龟	多海龟生成 + 追踪第一只
CP4	Action — 长时间移动 + 抓捕服务	完整游戏循环
CP5	Parameter — 可配置行为	启动时配置 + 运行时查看
CP6	Launch — 一键启动	完整系统交付

工作空间初始化

在开始 Checkpoint 之前，创建工作空间目录结构：

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws/src
```

Checkpoint 1-2 给出完整最小代码；后续只列出相对上一步的增量修改，最终版本以仓库源码为准。这样既能按顺序练习，也避免同一文件被重复粘贴多次。

讲次 4: 🎯 Checkpoint 1 — 创建节点, 发布 Topic 控制海龟运动

目标

- 创建 Python 功能包 `turtlesim_catch_them_all`
- 编写 `turtle_controller` 节点, 通过 Topic 发布速度指令
- 实现开环控制: 海龟以恒定速度画圆

本 Checkpoint 重点: Topic Publisher (2.2 知识)

Step 1: 创建功能包

```
cd ~/ros2_ws/src
ros2 pkg create turtlesim_catch_them_all --build-type ament_python --dependencies rclpy geometry
```

Step 2: 编写控制器节点

创建 `turtlesim_catch_them_all/turtlesim_catch_them_all/turtle_controller.py` :

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class TurtleControllerNode(Node):
    def __init__(self):
        super().__init__("turtle_controller")
        self.cmd_vel_publisher_ = self.create_publisher(
            Twist, "/turtle1/cmd_vel", 10)
        self.control_loop_timer_ = self.create_timer(
            0.01, self.control_loop)
        self.get_logger().info("Turtle Controller started - open loop mode")

    def control_loop(self):
        cmd = Twist()
        cmd.linear.x = 1.0    # 前进速度 1.0 m/s (Turtlesim 使用抽象单位, 非真实物理单位 m/s)
        cmd.angular.z = 0.5   # 旋转速度 0.5 rad/s
        self.cmd_vel_publisher_.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = TurtleControllerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        stop = Twist()
        node.cmd_vel_publisher_.publish(stop)
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

Step 3: 配置入口点

编辑 `setup.py` , 添加入口点:

```
entry_points={
    'console_scripts': [
        'controller = turtlesim_catch_them_all.turtle_controller:main',
    ],
},
```

Step 4: 构建与运行

```
cd ~/ros2_ws
colcon build --packages-select turtlesim_catch_them_all --symlink-install
source install/setup.bash
```

```
# 终端 1
ros2 run turtlesim turtlesim_node

# 终端 2
ros2 run turtlesim_catch_them_all controller
```

验证：海龟做圆周运动。

```
# 终端 3 - 确认话题
ros2 topic info /turtle1/cmd_vel
```

C++ 关键差异

项目	Python (rclpy)	C++ (rclcpp)
创建 Publisher	<code>create_publisher(Twist, "topic", 10)</code>	<code>create_publisher<geometry_msgs::msg::Twist>("topic", 10)</code>
定时器	<code>create_timer(0.01, callback)</code>	<code>create_wall_timer(10ms, std::bind(...))</code>
消息构造	<code>cmd = Twist(); cmd.linear.x = 1.0</code>	<code>auto cmd = geometry_msgs::msg::Twist(); cmd.linear.x = 1.0;</code>
头文件	无需	<code>#include "geometry_msgs/msg/twist.hpp"</code>

讲次 5: ❏ Checkpoint 2 — 添加 Subscriber, 实现状态监听

目标

- 订阅 `/turtle1/pose`, 获取海龟实时位姿
- 实现闭环控制: 自动导航到固定目标点 (2.0, 8.0)
- 用定时器驱动控制回路 (100 Hz)

本 Checkpoint 重点: Topic **Subscriber** + 闭环控制 (2.2 知识)

完整代码

替换 `turtle_controller.py` 为以下内容:

```

#!/usr/bin/env python3
import math
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose
from geometry_msgs.msg import Twist

class TurtleControllerNode(Node):
    def __init__(self):
        super().__init__("turtle_controller")
        self.target_x_ = 2.0
        self.target_y_ = 8.0
        self.pose_ = None
        self.cmd_vel_publisher_ = self.create_publisher(
            Twist, "/turtle1/cmd_vel", 10)
        self.pose_subscriber_ = self.create_subscription(
            Pose, "/turtle1/pose", self.callback_pose, 10)
        self.control_loop_timer_ = self.create_timer(
            0.01, self.control_loop)
        self.get_logger().info(
            f"Target: ({self.target_x_}, {self.target_y_})")

    def callback_pose(self, pose: Pose):
        self.pose_ = pose

    def control_loop(self):
        if self.pose_ is None:
            return

        dist_x = self.target_x_ - self.pose_.x
        dist_y = self.target_y_ - self.pose_.y
        distance = math.sqrt(dist_x * dist_x + dist_y * dist_y)

        cmd = Twist()

        if distance > 0.5:
            cmd.linear.x = min(2.0 * distance, 2.0)
            goal_theta = math.atan2(dist_y, dist_x)
            diff = goal_theta - self.pose_.theta
            if diff > math.pi:
                diff -= 2.0 * math.pi
            elif diff < -math.pi:
                diff += 2.0 * math.pi
            cmd.angular.z = 6.0 * diff
        else:
            cmd.linear.x = 0.0
            cmd.angular.z = 0.0
            self.get_logger().info("Target reached!")

        self.cmd_vel_publisher_.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = TurtleControllerNode()
    try:

```

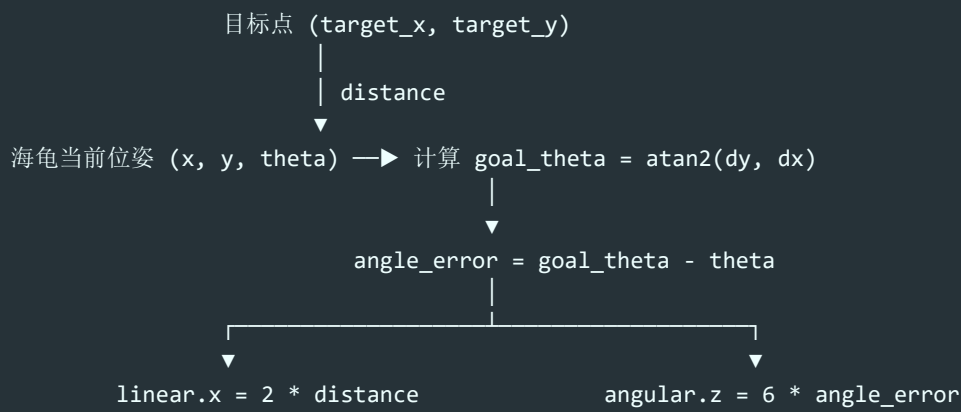
```

    rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    stop = Twist()
    node.cmd_vel_publisher_.publish(stop)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

控制算法说明



- 距离 > 0.5: 发布速度指令, 朝目标移动
- 距离 ≤ 0.5: 停止运动, 判定到达

构建与验证

```

cd ~/ros2_ws
colcon build --packages-select turtlesim_catch_them_all --symlink-install
source install/setup.bash

# 终端 1
ros2 run turtlesim turtlesim_node

# 终端 2
ros2 run turtlesim_catch_them_all controller

```

验证: 海龟自动移动到左上角区域 (2, 8) 并停止, 终端输出 Target reached!

```

# 终端 3 - 确认订阅关系
ros2 topic info /turtle1/pose -v

```

C++ 关键差异

```
// 订阅回调使用 SharedPtr
void callbackPose(const turtlesim::msg::Pose::SharedPtr pose)
{
    pose_ = *pose;
}

// 创建订阅
pose_subscriber_ = this->create_subscription<turtlesim::msg::Pose>(
    "/turtle1/pose", 10,
    std::bind(&TurtleControllerNode::callbackPose, this, std::placeholders::_1));
```

讲次 6: 🎯 Checkpoint 3 — 自定义接口 + Service Client 生成多海龟

目标

- 创建接口包 `my_robot_interfaces` , 定义 `Turtle.msg` 和 `TurtleArray.msg`
- 编写 `turtle_spawner` 节点, 调用 `/spawn` 服务在随机位置生成海龟
- 发布 `alive_turtles` 话题, 广播存活海龟列表
- 更新 `turtle_controller` , 订阅 `alive_turtles` 并追踪第一只海龟

本 Checkpoint 重点: Service Client + 自定义 Msg + 跨节点 Topic 协作 (2.3 + 2.5 知识)

Step 1: 创建接口包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake my_robot_interfaces
```

创建 `my_robot_interfaces/msg/Turtle.msg` :

```
string name
float32 x
float32 y
float32 theta
```

与 `turtlesim/msg/Pose` 保持一致, 使用 `float32` (`turtlesim/srv/Spawn` 的坐标也是 `float32`)。

创建 `my_robot_interfaces/msg/TurtleArray.msg` :

```
Turtle[] turtles
```

编辑 `my_robot_interfaces/CMakeLists.txt` :

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Turtle.msg"
  "msg/TurtleArray.msg"
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

编辑 `my_robot_interfaces/package.xml` , 在 `<buildtool_depend>ament_cmake</buildtool_depend>` 后添加:

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

Step 2: 编写 spawner 节点

创建 `turtlesim_catch_them_all/turtlesim_catch_them_all/turtle_spawner.py` :

```

#!/usr/bin/env python3
import math
import random
import rclpy
from functools import partial
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from turtlesim.srv import Spawn
from my_robot_interfaces.msg import Turtle, TurtleArray

class TurtleSpawnerNode(Node):
    def __init__(self):
        super().__init__("turtle_spawner")
        self.turtle_name_prefix_ = "turtle"
        self.turtle_counter_ = 1 # turtle1 是默认主海龟, 从 2 开始
        self.alive_turtles_ = []
        self.alive_turtles_publisher_ = self.create_publisher(
            TurtleArray, "alive_turtles", 10)
        self.spawn_client_ = self.create_client(Spawn, "/spawn")
        self.spawn_turtle_timer_ = self.create_timer(
            2.0, self.spawn_new_turtle)
        self.get_logger().info("Turtle Spawner started")

    def publish_alive_turtles(self):
        msg = TurtleArray()
        msg.turtles = self.alive_turtles_
        self.alive_turtles_publisher_.publish(msg)

    def spawn_new_turtle(self):
        self.turtle_counter_ += 1
        name = self.turtle_name_prefix_ + str(self.turtle_counter_)
        x = random.uniform(1.0, 10.0) # 避开边界, 防止海龟贴墙无法接近
        y = random.uniform(1.0, 10.0)
        theta = random.uniform(0.0, 2.0 * math.pi)
        self.call_spawn_service(name, x, y, theta)

    def call_spawn_service(self, turtle_name, x, y, theta):
        if not self.spawn_client_.wait_for_service(timeout_sec=2.0):
            self.get_logger().error("/spawn service is not available")
            return

        request = Spawn.Request()
        request.x = x
        request.y = y
        request.theta = theta
        request.name = turtle_name

        future = self.spawn_client_.call_async(request)
        future.add_done_callback(
            partial(self.callback_call_spawn_service, request=request))

    def callback_call_spawn_service(self, future, request: Spawn.Request):
        response = future.result()
        if response.name != "":
            self.get_logger().info("New alive turtle: " + response.name)
            new_turtle = Turtle()

```

```

        new_turtle.name = response.name
        new_turtle.x = request.x
        new_turtle.y = request.y
        new_turtle.theta = request.theta
        self.alive_turtles_.append(new_turtle)
        self.publish_alive_turtles()

def main(args=None):
    rclpy.init(args=args)
    node = TurtleSpawnerNode()
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

Step 3: 更新 controller 节点

基于 Checkpoint 2 的 `turtle_controller.py` 修改。先删除固定的 `target_x_`、`target_y_`，导入自定义消息并保存当前目标：

```

from my_robot_interfaces.msg import Turtle, TurtleArray

# __init__ 中新增
self.turtle_to_catch_: Turtle | None = None
self.alive_turtles_subscriber_ = self.create_subscription(
    TurtleArray, "alive_turtles", self.callback_alive_turtles, 10)

def callback_alive_turtles(self, msg: TurtleArray):
    self.turtle_to_catch_ = msg.turtles[0] if msg.turtles else None

```

`control_loop()` 的守卫和目标差值改为：

```

if self.pose_ is None or self.turtle_to_catch_ is None:
    return

dist_x = self.turtle_to_catch_.x - self.pose_.x
dist_y = self.turtle_to_catch_.y - self.pose_.y

```

其余转向与速度计算沿用 Checkpoint 2；到达后只打印目标名，此时还没有实现 `/kill`：

```
if distance <= 0.5:
    cmd = Twist()
    self.get_logger().info(
        f'Reached {self.turtle_to_catch_.name} (not removed yet)')
```

Step 4: 更新包配置

turtlesim_catch_them_all/package.xml 添加依赖:

```
<depend>my_robot_interfaces</depend>
```

setup.py 添加入口点:

```
entry_points={
    'console_scripts': [
        'controller = turtlesim_catch_them_all.turtle_controller:main',
        'spawner = turtlesim_catch_them_all.turtle_spawner:main',
    ],
},
```

Step 5: 构建与验证

```
cd ~/ros2_ws
colcon build --packages-select my_robot_interfaces turtlesim_catch_them_all --symlink-install
source install/setup.bash
```

```
# 终端 1
ros2 run turtlesim turtlesim_node

# 终端 2
ros2 run turtlesim_catch_them_all spawner

# 终端 3
ros2 run turtlesim_catch_them_all controller
```

验证:

- 每 2 秒生成一只新海龟
- `turtle1` 追踪列表中第一只海龟
- `ros2 topic echo /alive_turtles` 可看到海龟列表

```
ros2 service list -t | grep spawn
ros2 topic info /alive_turtles
```

C++ 关键差异

```
// 自定义消息头文件（自动生成）
#include "my_robot_interfaces/msg/turtle.hpp"
#include "my_robot_interfaces/msg/turtle_array.hpp"

// Service 异步调用
auto request = std::make_shared<turtlesim::srv::Spawn::Request>();
request->x = x;
auto result_future = spawn_client_->async_send_request(request);
```

讲次 7: 🎯 Checkpoint 4 — 添加 Action, 实现长时间任务管理

目标

- 定义 `MoveToPosition.action` 和 `CatchTurtle.srv` 自定义接口
- 编写 `move_to_position_server`, 用 Action Server 管理移动任务
- 重构 `turtle_controller` 为 Action Client, 发送移动目标
- 在 `turtle_spawner` 中实现 `catch_turtle` 服务, 调用 `/kill` 消除海龟
- 完成完整游戏循环: 追踪 → 到达 → 抓捕 → 生成新海龟

本 Checkpoint 重点: Action Server/Client + 自定义 Srv (2.4 + 2.3 知识)

Step 1: 扩展接口包

创建 `my_robot_interfaces/srv/CatchTurtle.srv` :

```
string name
---
bool success
```

创建 `my_robot_interfaces/action/MoveToPosition.action` :

```
# Goal
float32 target_x
float32 target_y
---
# Result
float32 final_x
float32 final_y
float32 distance_error
---
# Feedback
float32 current_x
float32 current_y
float32 distance_remaining
```

更新 `my_robot_interfaces/CMakeLists.txt` :

```

find_package(rosidl_default_generators REQUIRED)
find_package(action_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Turtle.msg"
  "msg/TurtleArray.msg"
  "srv/CatchTurtle.srv"
  "action/MoveToPosition.action"
  DEPENDENCIES action_msgs
)

```

关键： `DEPENDENCIES action_msgs` 告诉 rosidl 生成器 Action 依赖 `action_msgs` 包中的类型（如 `GoalStatus` ），但必须先有对应的 `find_package(action_msgs REQUIRED)` 才能找到它。缺少会导致 CMake 报错。

`package.xml` 添加：

```
<depend>action_msgs</depend>
```

Step 2: 编写 Action Server

创建 `turtlesim_catch_them_all/turtlesim_catch_them_all/move_to_position_server.py`：


```

#!/usr/bin/env python3
import math
import time

import rclpy
from geometry_msgs.msg import Twist
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import ExternalShutdownException, MultiThreadedExecutor
from rclpy.node import Node
from turtlesim.msg import Pose

from my_robot_interfaces.action import MoveToPosition

class MoveToPositionServer(Node):
    def __init__(self):
        super().__init__("move_to_position_server")
        self.current_pose_ = Pose()
        self.pose_subscriber_ = self.create_subscription(
            Pose, "/turtle1/pose", self.callback_pose, 10)
        self.cmd_vel_publisher_ = self.create_publisher(
            Twist, "/turtle1/cmd_vel", 10)
        self.action_server_ = ActionServer(
            self,
            MoveToPosition,
            "move_to_position",
            self.execute_callback,
            callback_group=ReentrantCallbackGroup(),
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
        )
        self.get_logger().info("MoveToPosition Action Server ready")

    def callback_pose(self, msg: Pose):
        self.current_pose_ = msg

    def goal_callback(self, goal_request):
        self.get_logger().info(
            f"Goal received: ({goal_request.target_x:.2f}, "
            f"{goal_request.target_y:.2f})")
        return GoalResponse.ACCEPT

    def cancel_callback(self, goal_handle):
        self.get_logger().info("Cancel accepted")
        self.cmd_vel_publisher_.publish(Twist())
        return CancelResponse.ACCEPT

    def execute_callback(self, goal_handle):
        target_x = goal_handle.request.target_x
        target_y = goal_handle.request.target_y
        tolerance = 0.5
        max_speed = 2.0
        feedback = MoveToPosition.Feedback()
        result = MoveToPosition.Result()

        while rclpy.ok():

```

```

        if goal_handle.is_cancel_requested:
            self.cmd_vel_publisher_.publish(Twist())
            goal_handle.canceled()
            result.final_x = self.current_pose_.x
            result.final_y = self.current_pose_.y
            result.distance_error = self.compute_distance(
                target_x, target_y)
            return result

        dx = target_x - self.current_pose_.x
        dy = target_y - self.current_pose_.y
        distance = math.sqrt(dx * dx + dy * dy)

        feedback.current_x = self.current_pose_.x
        feedback.current_y = self.current_pose_.y
        feedback.distance_remaining = distance
        goal_handle.publish_feedback(feedback)

        if distance < tolerance:
            self.cmd_vel_publisher_.publish(Twist())
            result.final_x = self.current_pose_.x
            result.final_y = self.current_pose_.y
            result.distance_error = distance
            goal_handle.succeed()
            self.get_logger().info(
                f"Arrived! Error: {distance:.3f}")
            return result

        angle_to_goal = math.atan2(dy, dx)
        angle_error = self.normalize_angle(
            angle_to_goal - self.current_pose_.theta)

        cmd = Twist()
        cmd.linear.x = min(max_speed, 2.0 * distance)
        cmd.angular.z = 6.0 * angle_error
        self.cmd_vel_publisher_.publish(cmd)
        time.sleep(0.05)

    def compute_distance(self, target_x, target_y):
        dx = target_x - self.current_pose_.x
        dy = target_y - self.current_pose_.y
        return math.sqrt(dx * dx + dy * dy)

    @staticmethod
    def normalize_angle(angle):
        while angle > math.pi:
            angle -= 2.0 * math.pi
        while angle < -math.pi:
            angle += 2.0 * math.pi
        return angle

def main(args=None):
    rclpy.init(args=args)
    node = MoveToPositionServer()
    executor = MultiThreadedExecutor()
    executor.add_node(node)

```

```

try:
    executor.spin()
except (KeyboardInterrupt, ExternalShutdownException):
    pass
finally:
    executor.shutdown(timeout_sec=2.0)
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

以上代码突出 Goal/Feedback/Result 与取消流程。仓库中的最终参考实现还增加了目标范围校验、等待首帧 Pose（初值为 `None` 而非 `Pose()`）、速度参数化（`max_speed`）、关机停车、活动 Goal 收尾，以及“已有移动任务时拒绝第二个 Goal”的互斥保护；到位阈值也从教学版的 0.5 收紧为 0.2。计数 Action 可以并发执行，但同一台机器人不能让多个控制循环同时发布速度；项目交付应以该加固版本为准。

Step 3: 更新 controller (Action Client)

替换 `turtle_controller.py`：

```

#!/usr/bin/env python3
import rclpy
from functools import partial
from action_msgs.msg import GoalStatus
from rclpy.action import ActionClient
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from turtlesim.msg import Pose
from my_robot_interfaces.msg import Turtle, TurtleArray
from my_robot_interfaces.action import MoveToPosition
from my_robot_interfaces.srv import CatchTurtle

class TurtleControllerNode(Node):
    def __init__(self):
        super().__init__("turtle_controller")
        self.turtle_to_catch_ = None
        self.pose_ = None
        self.goal_in_progress_ = False
        self.pose_subscriber_ = self.create_subscription(
            Pose, "/turtle1/pose", self.callback_pose, 10)
        self.alive_turtles_subscriber_ = self.create_subscription(
            TurtleArray, "alive_turtles", self.callback_alive_turtles, 10)
        self.move_action_client_ = ActionClient(
            self, MoveToPosition, "move_to_position")
        self.catch_turtle_client_ = self.create_client(
            CatchTurtle, "catch_turtle")
        self.get_logger().info("Turtle Controller started – Action mode")

    def callback_pose(self, pose: Pose):
        self.pose_ = pose

    def callback_alive_turtles(self, msg: TurtleArray):
        if len(msg.turtles) > 0 and not self.goal_in_progress_:
            self.turtle_to_catch_ = msg.turtles[0]
            self.send_move_goal(
                self.turtle_to_catch_.x, self.turtle_to_catch_.y)

    def send_move_goal(self, x, y):
        self.goal_in_progress_ = True
        if not self.move_action_client_.wait_for_server(timeout_sec=5.0):
            self.get_logger().error("Action server not available, retrying later")
            self.goal_in_progress_ = False
            return
        goal = MoveToPosition.Goal()
        goal.target_x = float(x)
        goal.target_y = float(y)
        self.get_logger().info(f"Sending move goal: ({x:.2f}, {y:.2f})")
        future = self.move_action_client_.send_goal_async(
            goal, feedback_callback=self.feedback_callback)
        future.add_done_callback(self.goal_response_callback)

    def feedback_callback(self, feedback_msg):
        fb = feedback_msg.feedback
        self.get_logger().info(
            f"Distance remaining: {fb.distance_remaining:.2f}",
            throttle_duration_sec=1.0)

```

```

def goal_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().warn("Goal rejected")
        self.goal_in_progress_ = False
        return
    self.goal_handle_ = goal_handle
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def result_callback(self, future):
    try:
        wrapped_result = future.result()
        if wrapped_result.status != GoalStatus.STATUS_SUCCEEDED:
            self.get_logger().warn(
                f"Move did not succeed; status={wrapped_result.status}")
            self.turtle_to_catch_ = None
            self.goal_in_progress_ = False
            return
        result = wrapped_result.result
        self.get_logger().info(
            f"Move done. Error: {result.distance_error:.3f}")
    except Exception as e:
        self.get_logger().error(f"Action failed: {e}")
        self.turtle_to_catch_ = None
        self.goal_in_progress_ = False
        return
    if self.turtle_to_catch_ is not None:
        self.call_catch_turtle_service(self.turtle_to_catch_.name)

def call_catch_turtle_service(self, turtle_name):
    if not self.catch_turtle_client_.wait_for_service(timeout_sec=2.0):
        self.get_logger().error("catch_turtle service is not available")
        self.turtle_to_catch_ = None
        self.goal_in_progress_ = False
        return
    request = CatchTurtle.Request()
    request.name = turtle_name
    future = self.catch_turtle_client_.call_async(request)
    future.add_done_callback(
        partial(self.callback_catch_turtle, turtle_name=turtle_name))

def callback_catch_turtle(self, future, turtle_name):
    try:
        response = future.result()
        if response.success:
            self.get_logger().info(f"Caught turtle: {turtle_name}")
        else:
            self.get_logger().error(f"Failed to catch turtle: {turtle_name}")
    finally:
        self.turtle_to_catch_ = None
        self.goal_in_progress_ = False

def main(args=None):
    rclpy.init(args=args)

```

```

node = TurtleControllerNode()
try:
    rclpy.spin(node)
except (KeyboardInterrupt, ExternalShutdownException):
    pass
finally:
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

Step 4: 更新 spawner (抓捕服务 + Kill)

沿用 Checkpoint 3 的 `turtle_spawner.py` , 新增 Kill Client 和 Catch Service:

```

from turtlesim.srv import Kill, Spawn
from my_robot_interfaces.srv import CatchTurtle

# __init__ 中新增
self.kill_client_ = self.create_client(Kill, "/kill")
self.catch_turtle_service_ = self.create_service(
    CatchTurtle, "catch_turtle", self.callback_catch_turtle)

```

抓捕回调先确认 Kill 请求可以排队, 再更新 `alive_turtles` 。这样 `/kill` 不可用时, 内部列表不会与 Turtlesim 画面失去一致性:

```

def callback_catch_turtle(self, request, response):
    for index, turtle in enumerate(self.alive_turtles_):
        if turtle.name == request.name:
            if not self.call_kill_service(request.name):
                response.success = False
                return response
            del self.alive_turtles_[index]
            self.publish_alive_turtles()
            response.success = True
            return response

    response.success = False
    return response

def call_kill_service(self, turtle_name):
    if not self.kill_client_.wait_for_service(timeout_sec=2.0):
        self.get_logger().error("/kill service is not available")
        return False

    request = Kill.Request()
    request.name = turtle_name
    future = self.kill_client_.call_async(request)
    future.add_done_callback(
        partial(self.callback_kill_done, turtle_name=turtle_name))
    return True

def callback_kill_done(self, future, turtle_name):
    try:
        future.result()
        self.get_logger().info(f"Killed turtle: {turtle_name}")
    except Exception as exc:
        self.get_logger().error(f"Failed to kill {turtle_name}: {exc}")

```

这里的 `success=True` 表示 Kill 请求已成功提交；实际异步调用若失败，会由 `callback_kill_done()` 记录错误。最终完整文件见

`stage2_code/turtlesim_catch_them_all/turtlesim_catch_them_all/turtle_spawner.py`。

Step 5: 更新包配置

`setup.py` 添加入口点:

```
'move_to_position_server' = turtlesim_catch_them_all.move_to_position_server:main',
```

`package.xml` 确认已有:

```
<depend>rclpy</depend>
<depend>action_msgs</depend>
```

Python 的 Action Client/Server API 由 `rclpy` 提供；本例还直接导入 `action_msgs/msg/GoalStatus` 判断结果状态，因此应用包必须声明 `action_msgs`。C++ 版本使用 Action API 时依赖 `rclcpp_action`。

Step 6: 构建与验证

```
cd ~/ros2_ws
colcon build --packages-select my_robot_interfaces turtlesim_catch_them_all --symlink-install
source install/setup.bash
```

```
# 终端 1
ros2 run turtlesim turtlesim_node

# 终端 2
ros2 run turtlesim_catch_them_all move_to_position_server

# 终端 3
ros2 run turtlesim_catch_them_all spawner

# 终端 4
ros2 run turtlesim_catch_them_all controller
```

验证:

- 海龟生成 → 主海龟追踪 → 到达后消失 → 循环继续
- Action 反馈日志显示剩余距离

```
ros2 action list -t
ros2 action info /move_to_position -v
ros2 service type /catch_turtle
```

测试 Action 取消 (可选) :

```
ros2 action send_goal /move_to_position my_robot_interfaces/action/MoveToPosition \
  "{target_x: 9.0, target_y: 9.0}" --feedback
# 中途 Ctrl+C 取消
```

C++ 关键差异

项目	Python	C++
Action Server	<code>ActionServer(self, Type, "name", execute_cb, ...)</code>	<code>rclcpp_action::create_server<Type>(...)</code>
Action Client	<code>ActionClient(self, Type, "name")</code>	<code>rclcpp_action::create_client<Type>(...)</code>
执行器	<code>MultiThreadedExecutor</code> (Action Server 必须)	<code>rclcpp::executors::MultiThreadedExecutor</code>
头文件	<code>from rclpy.action import ActionServer</code>	<code>#include "rclcpp_action/rclcpp_action.hpp"</code>

讲次 8: 🎯 Checkpoint 5 — 添加 Parameter, 实现可配置行为

目标

- 在 `turtle_controller` 中添加 `catch_closest_turtle_first` 参数
- 在 `turtle_spawner` 中添加 `spawn_frequency`、`turtle_name_prefix` 和 `max_alive_turtles` 参数
- 在 `move_to_position_server` 中添加 `max_speed` 参数
- 通过 YAML 文件和命令行配置参数, 理解启动时参数覆盖机制

本 Checkpoint 重点: `declare_parameter` / `get_parameter` (2.6 知识)

增量修改

本 Checkpoint 基于 CP4 的完整代码, 只展示需要修改的部分, 避免重复整份文件。

`move_to_position_server.py`: 在 `__init__` 中声明并读取速度参数:

```
self.declare_parameter("max_speed", 2.0)
self.max_speed_ = self.get_parameter("max_speed").value
```

在 `execute_callback` 中删除原来的硬编码:

```
max_speed = 2.0
```

改为:

```
max_speed = self.max_speed_
```

`turtle_controller.py`: 新增 `math` 导入 (如果 CP4 版本没有) :

```
import math
```

在 `__init__` 中声明追踪策略参数, 并读取到成员变量:

```
self.declare_parameter("catch_closest_turtle_first", True)
self.catch_closest_turtle_first_ = self.get_parameter(
    "catch_closest_turtle_first").value
```

将 CP4 的 `callback_alive_turtles` 替换为:

```

def callback_alive_turtles(self, msg: TurtleArray):
    if len(msg.turtles) == 0 or self.goal_in_progress_:
        return
    if self.pose_ is None:
        return

    if self.catch_closest_turtle_first_:
        closest = None
        closest_dist = None
        for turtle in msg.turtles:
            dx = turtle.x - self.pose_.x
            dy = turtle.y - self.pose_.y
            dist = math.sqrt(dx * dx + dy * dy)
            if closest is None or dist < closest_dist:
                closest = turtle
                closest_dist = dist
        self.turtle_to_catch_ = closest
    else:
        self.turtle_to_catch_ = msg.turtles[0]

    self.send_move_goal(
        self.turtle_to_catch_.x, self.turtle_to_catch_.y)

```

turtle_spawner.py: 在 `__init__` 中声明并读取生成参数:

```

self.declare_parameter("turtle_name_prefix", "turtle")
self.declare_parameter("spawn_frequency", 1.0)
self.declare_parameter("max_alive_turtles", 8)

self.turtle_name_prefix_ = self.get_parameter(
    "turtle_name_prefix").value
self.spawn_frequency_ = self.get_parameter(
    "spawn_frequency").value
self.max_alive_turtles_ = self.get_parameter(
    "max_alive_turtles").value

```

将定时器周期从固定 `2.0` 替换为参数控制:

```

self.spawn_turtle_timer_ = self.create_timer(
    1.0 / self.spawn_frequency_, self.spawn_new_turtle)

```

`spawn_new_turtle()` 中继续使用 `self.turtle_name_prefix_` 生成名称:

```

if len(self.alive_turtles_) >= self.max_alive_turtles_:
    return

name = self.turtle_name_prefix_ + str(self.turtle_counter_)

```

运行时查看参数

```
# 查看参数
ros2 param list /turtle_spawner
ros2 param get /turtle_spawner spawn_frequency
ros2 param get /turtle_spawner max_alive_turtles

# 尝试运行时修改（注意：当前代码仅在启动时读取参数，运行时修改不会立即改变定时器）
ros2 param set /turtle_spawner spawn_frequency 0.5
# 参数值已更新，但 timer 频率不变。需要 add_on_set_parameters_callback 才能动态重建 timer。

# 同理，catch_closest_turtle_first 当前也是启动时读取；运行时 set 只验证参数服务可用
ros2 param set /turtle_controller catch_closest_turtle_first false
```

构建与验证

```
cd ~/ros2_ws
colcon build --packages-select turtlesim_catch_them_all --symlink-install
source install/setup.bash
```

分别启动 4 个节点（同 CP4），观察：

- 默认追踪最近的海龟
- 若要改为追踪列表第一只，可在启动时通过 YAML 或 `--ros-args -p catch_closest_turtle_first:=false` 覆盖参数

C++ 关键差异

```
// 声明与读取
this->declare_parameter("spawn_frequency", 1.0);
double freq = this->get_parameter("spawn_frequency").as_double();

// 定时器周期
auto period = std::chrono::duration<double>(1.0 / freq);
```

讲次 9: 🎯 Checkpoint 6 — 整合 Launch 文件，一键启动完整系统

目标

- 创建 `my_robot_bringup` 功能包
- 编写 YAML 参数文件
- 编写 XML Launch 文件，一键启动全部 4 个节点
- 实现完整系统的工程化交付

本 Checkpoint 重点： Launch + YAML 参数集成 (2.6 知识)

Step 1: 创建 bringup 包

```
cd ~/ros2_ws/src
ros2 pkg create my_robot_bringup --build-type ament_cmake
```

创建目录：

```
mkdir -p my_robot_bringup/launch my_robot_bringup/config
```

Step 2: YAML 参数文件

创建 `my_robot_bringup/config/catch_them_all_config.yaml`：

```
/turtle_controller:
  ros__parameters:
    catch_closest_turtle_first: true

/turtle_spawner:
  ros__parameters:
    turtle_name_prefix: "my_turtle"
    spawn_frequency: 1.5
    max_alive_turtles: 8

/move_to_position_server:
  ros__parameters:
    max_speed: 2.0
```

Step 3: Launch 文件

创建 `my_robot_bringup/launch/turtlesim_catch_them_all.launch.xml`：

```

<launch>
  <!-- Turtlesim 仿真器 -->
  <node pkg="turtlesim" exec="turtlesim_node" />

  <!-- Action Server: 管理移动任务 -->
  <node pkg="turtlesim_catch_them_all" exec="move_to_position_server">
    <param from="$(find-pkg-share my_robot_bringup)/config/catch_them_all_config.yaml" />
  </node>

  <!-- 控制器: Action Client + 抓捕逻辑 -->
  <node pkg="turtlesim_catch_them_all" exec="controller">
    <param from="$(find-pkg-share my_robot_bringup)/config/catch_them_all_config.yaml" />
  </node>

  <!-- 生成器: 定时生成 + 抓捕服务 -->
  <node pkg="turtlesim_catch_them_all" exec="spawner">
    <param from="$(find-pkg-share my_robot_bringup)/config/catch_them_all_config.yaml" />
  </node>

</launch>

```

说明： 本项目使用 XML Launch 文件作为最终交付格式。XML 格式在简单的节点组合场景中更简洁，且与 ROS 2 生态的 `launch.xml` 标准兼容。在需要条件判断或复杂逻辑时，应使用 Python `.launch.py` 格式（参见 2.6 节）。

Step 4: 配置 bringup 包

`my_robot_bringup/CMakeLists.txt` :

```

cmake_minimum_required(VERSION 3.8)
project(my_robot_bringup)

find_package(ament_cmake REQUIRED)

install(DIRECTORY launch config
  DESTINATION share/${PROJECT_NAME}/
)

ament_package()

```

`my_robot_bringup/package.xml` :

```
<?xml version="1.0"?>
<package format="3">
  <name>my_robot_bringup</name>
  <version>0.0.0</version>
  <description>Launch files for Catch Them All project</description>
  <maintainer email="student@todo.todo">student</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>turtlesim</exec_depend>
  <exec_depend>turtlesim_catch_them_all</exec_depend>
  <exec_depend>my_robot_interfaces</exec_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

Step 5: 构建与一键启动

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash

ros2 launch my_robot_bringup turtlesim_catch_them_all.launch.xml
```

验证清单:

检查项	命令	预期
节点全部启动	<code>ros2 node list</code>	含 turtlesim, turtle_controller, turtle_spawner, move_to_position_server
参数已加载	<code>ros2 param get /turtle_spawner spawn_frequency</code>	1.5
数量上限已加载	<code>ros2 param get /turtle_spawner max_alive_turtles</code>	8
游戏运行	观察窗口	海龟生成、追踪、抓捕循环
Action 活跃	<code>ros2 action list</code>	/move_to_position
自定义服务	<code>ros2 service list \ grep catch</code>	/catch_turtle

修改 YAML 后重新 launch 即可生效，无需改代码。



讲次 10: P1 交付 — 完整的多节点 ROS 2 应用

交付标准

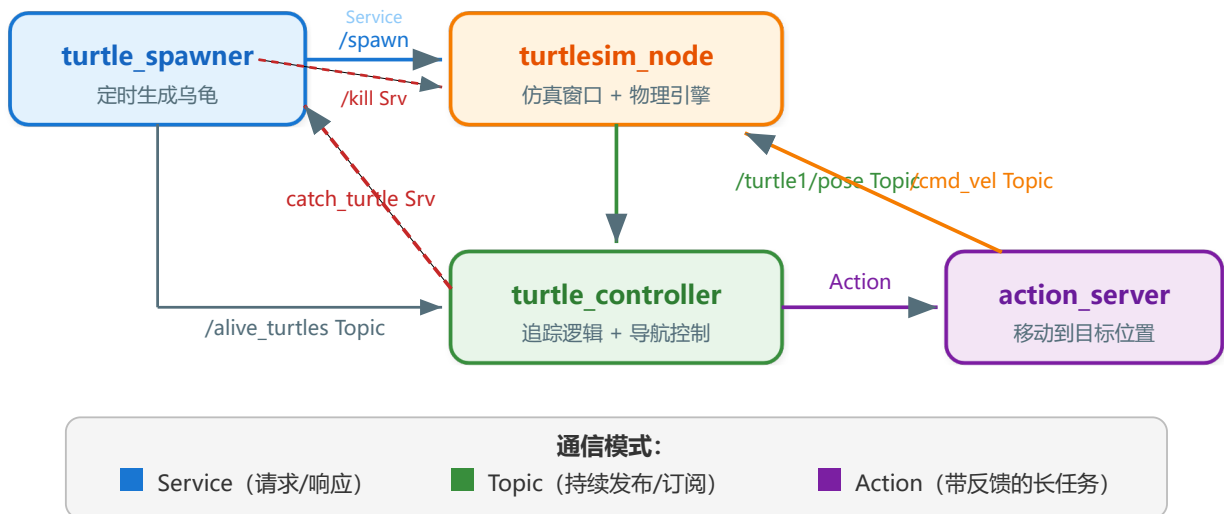
类别	要求	状态
功能包	my_robot_interfaces 、 turtlesim_catch_them_all 、 my_robot_bringup	✓
自定义接口	Turtle.msg 、 TurtleArray.msg 、 CatchTurtle.srv 、 MoveToPosition.action	✓
节点	controller、 spawner、 move_to_position_server + turtlesim_node	✓
通信机制	Topic + Service + Action + Parameter	✓
启动方式	ros2 launch my_robot_bringup turtlesim_catch_them_all.launch.xml	✓
可配置	YAML 参数文件控制行为	✓

最终项目结构

```
ros2_ws/src/
├── my_robot_interfaces/                                # 接口包
│   ├── msg/
│   │   ├── Turtle.msg
│   │   └── TurtleArray.msg
│   ├── srv/
│   │   └── CatchTurtle.srv
│   ├── action/
│   │   └── MoveToPosition.action
│   ├── CMakeLists.txt
│   └── package.xml
├── turtlesim_catch_them_all/                          # 应用包 (Python)
│   ├── turtlesim_catch_them_all/
│   │   ├── turtle_controller.py                    # Action Client + 追踪逻辑
│   │   ├── turtle_spawner.py                      # 生成 + 抓捕服务
│   │   └── move_to_position_server.py              # Action Server
│   ├── setup.py
│   ├── setup.cfg
│   └── package.xml
└── my_robot_bringup/                                  # 启动包
    ├── launch/
    │   └── turtlesim_catch_them_all.launch.xml
    ├── config/
    │   └── catch_them_all_config.yaml
    ├── CMakeLists.txt
    └── package.xml
```

通信拓扑总览

Turtlesim Catch 通信拓扑



图：Turtlesim Catch 游戏通信拓扑 —— 综合运用 Topic / Service / Action 三种模式

自检命令

```
# 1. 完整构建
cd ~/ros2_ws && colcon build --symlink-install && source install/setup.bash

# 2. 一键启动
ros2 launch my_robot_bringup turtlesim_catch_them_all.launch.xml

# 3. 通信诊断
ros2 node list
ros2 topic list -t
ros2 service list -t
ros2 action list -t
ros2 param list

# 4. 接口验证
ros2 interface show my_robot_interfaces/msg/Turtle
ros2 interface show my_robot_interfaces/srv/CatchTurtle
ros2 interface show my_robot_interfaces/action/MoveToPosition
```

扩展挑战 (选做)

- 计分系统:** 添加 `score_publisher` 节点, 发布已抓捕海龟数量
- 难度递增:** 用 `Parameter` 实现生成频率随时间递增
- C++ 重写:** 将 Python 节点翻译为 C++, 在 `Launch` 中切换
- Python Launch:** 将 XML `Launch` 改写为 Python 版 (`launch.py`)
- 参数回调:** 实现 `add_on_set_parameters_callback`, 运行时动态调整速度

讲次 11：项目总结与回顾

技能总结

通过本项目，你已掌握以下实战技能：

技能	具体体现
多节点协作	4 个节点各司其职，通过 ROS 2 通信机制协同
自定义接口	定义 Msg/Srv/Action，构建接口包供多节点共享
Topic 双向通信	Publisher 控制运动 + Subscriber 监听状态
Service 请求响应	调用官方 <code>/spawn</code> <code>/kill</code> + 自定义 <code>catch_turtle</code>
Action 长任务管理	移动任务带 Feedback，支持取消
Parameter 配置	声明参数、YAML 加载、运行时修改
Launch 工程化	一键启动完整系统，参数与节点解耦
递进式开发	6 个 Checkpoint 逐步叠加，每步可独立验证

Stage 2 完整回顾

```
第二阶段：ROS 2 核心通信机制
├── 2.1 节点基础
│   └── 节点概念、OOP 模板、功能包创建、colcon 构建
├── 2.2 话题通信 ← CP1, CP2
│   └── Publisher/Subscriber、Twist/Pose、Turtlesim 画圆
├── 2.3 服务通信 ← CP3, CP4
│   └── Service Server/Client、/spawn、/kill、异步调用
├── 2.4 动作通信 ← CP4
│   └── Action Server/Client、Goal/Feedback/Result、取消机制
├── 2.5 自定义接口 ← CP3, CP4
│   └── .msg / .srv / .action 定义与构建
├── 2.6 Launch 与参数 ← CP5, CP6
│   └── XML/Python Launch、YAML 参数、重映射
└── 2.7 项目一 ← 本项目
    └── Catch Them All：五种机制综合实战
```

命令速查表

命令	说明
<code>ros2 run turtlesim turtlesim_node</code>	启动 Turtlesim
<code>ros2 topic echo /turtle1/pose</code>	查看海龟位姿
<code>ros2 topic pub /turtle1/cmd_vel ...</code>	手动控制运动
<code>ros2 service call /spawn turtlesim/srv/Spawn ...</code>	生成海龟
<code>ros2 service call /kill turtlesim/srv/Kill ...</code>	消除海龟
<code>ros2 action list -t</code>	列出 Action 及类型
<code>ros2 action info /move_to_position -v</code>	Action 详细信息
<code>ros2 param list</code>	列出节点参数
<code>ros2 param set <node> <param> <value></code>	运行时修改参数
<code>ros2 interface show my_robot_interfaces/msg/Turtle</code>	查看自定义接口
<code>ros2 launch my_robot_bringup turtlesim_catch_them_all.launch.xml</code>	一键启动
<code>colcon build --symlink-install</code>	构建工作空间

常见问题排查

现象	原因	解决
海龟不动	turtlesim_node 未启动	先启动仿真器
No module named 'my_robot_interfaces'	接口包未构建	<code>colcon build --packages-select my_robot_interfaces</code>
Action Server 无响应	未使用 MultiThreadedExecutor	检查 server 的 executor 配置
参数未生效	YAML 节点名不匹配	确保 YAML 中 <code>/node_name</code> 与代码一致
海龟生成但不停留	spawn 后未更新 alive_turtles	检查 spawner 回调逻辑
Launch 找不到文件	bringup 包未安装	<code>colcon build --packages-select my_robot_bringup</code>

下一步：Stage 3

恭喜完成 Stage 2 和项目一！接下来将进入 **第三阶段：坐标变换与机器人建模**，你将：

- 学习 **TF 坐标变换**，理解多传感器融合中的坐标系管理
- 使用 **URDF/Xacro** 描述机器人结构，构建移动机器人模型
- 在 **Gazebo** 中仿真真实物理环境
- 启动 **项目二：移动机器人仿真全栈项目**

```
# 预习：查看 TF 相关包  
apt list ros-jazzy-tf2*
```

讲师寄语： Turtlesim 虽小，却囊括了 ROS 2 通信的全部精髓。当你能在 Launch 文件中一键启动自己编写的多节点系统，并看着海龟自动追逐目标时——你已经从"ROS 2 学习者"成长为"ROS 2 开发者"。继续保持这种递进式、可验证的开发习惯，它将在后续 Gazebo 仿真和真机部署中发挥巨大价值。